DecodedSound

SDK / Esdeekid Music Translator

Full PRD + Engineering Spec

Version 3.2 — Single Source of Truth | February 2026

"Making the culture accessible to the world."

Genre: SDK / Esdeekid only (v1.0)

Stack: Next.js 14 · TypeScript · Prisma · Postgres · Vercel

AI: Gemini 2.5 Flash (translation) · Groq (detection + transcription)

Access: Fully open — no account required at launch

Team: 2 (Founder + AI)

Status: • Active Development

Contents

1 What We're Building & Why **3**

2 Design Principles **4**

3 Target Users **4**

4 Core Features **4**
- 4.1 Input Methods . . . 5
- 4.2 4-Panel Translation Output . . . 5
- 4.3 Slang Knowledge Base (KB) . . . 5
- 4.4 Public Song Library . . . 5
- 4.5 Sharing . . . 6

5 User Stories **6**
- 5.1 Typed Lyrics Input . . . 6
- 5.2 Audio Upload . . . 6
- 5.3 Streaming Link . . . 7
- 5.4 Translation Output . . . 7
- 5.5 Knowledge Base & Community . . . 9
- 5.6 Library & Social . . . 9

6 Error States **10**

7 Data Models **11**
- 7.1 Song . . . 11
- 7.2 Translation . . . 11
- 7.3 Job (Async Processing Queue) . . . 12
- 7.4 KBEntry . . . 12
- 7.5 KBCandidate . . . 12
- 7.6 Flag . . . 13
- 7.7 Rating . . . 13

8 API Routes **13**
- 8.1 Translation . . . 13
- 8.2 Songs . . . 14
- 8.3 Knowledge Base . . . 14
- 8.4 Ratings . . . 14

9 Folder Structure **14**

10 AI Pipeline — Step by Step **16**
- 10.1 Job Worker Design . . . 16
- 10.2 Gemini Translation Prompt . . . 17

11 AI Provider Strategy **18**
- 11.1 Why This Split . . . 18

11.2 Provider Abstraction Layer 19
11.3 When We'd Pay 19

12 Tech Stack — Final Decisions **20**

13 Environment Variables **21**

14 Milestone Plan **22**

15 MVP Scope **22**

16 Feature Spec: English → SDK Reverse Translation **24**
16.1 User Stories — Reverse Mode 24
16.2 How It Works — Reverse Pipeline 25
16.3 Gemini Prompt — Reverse Mode 25
16.4 DB Change for Reverse Mode 25
16.5 Competitive Advantage Over esdictionary.com 26

17 Success Metrics **26**

18 Risks & Mitigations **27**

19 Open Questions **27**

1. What We're Building & Why

DecodedSound is an AI-powered web platform that translates SDK (esdeekid) music into plain, accessible English — giving anyone in the world the tools to understand and connect with one of the most culturally rich street music genres to come out of South Africa.

SDK music is not just slang. It is a coded cultural language built on phonetic wordplay, Cape Flats dialect, Afrikaans and Xhosa borrowings, double meanings, and hyper-local references that no outsider can decode without help. No tool exists that does this well. We're building it.

DecodedSound takes audio files, streaming links, or typed lyrics and returns four things: a plain English translation, a line-by-line annotated breakdown, a slang dictionary, and a cultural context narrative.

Genre scope is SDK / esdeekid only for v1.0. We do one thing exceptionally well before expanding. Amapiano, Gqom, Kwaito are roadmapped for v2.0 only.

Document Metadata

Field	Value
Product Name	DecodedSound
Document Version	3.0 — Full PRD + Engineering Spec
Date	February 2026
Status	**Active Development**
Genre Scope v1.0	SDK / Esdeekid only
Genre Scope v2.0	+ Amapiano, Gqom, Kwaito
Access Model	Fully open, no account at launch
Platform v1.0	Web browser
Platform v2.0	+ Mobile (React Native)
UI Language	English (v1.0)
Team	2 (Founder + AI)
Monetization	TBD — decided before v1.5

2. Design Principles

These are not suggestions. Every decision in the product and in the code must hold up against these:

- **Cultural respect first** — we translate, we never police or sanitise. The goal is understanding, not judgment.
- **Accuracy over speed** — unknown terms are flagged, never guessed. Confidence is surfaced to the user.
- **Zero friction at entry** — no account, no signup wall, no paywall at launch. Land and translate immediately.
- **SDK focus, no exceptions in v1.0** — one genre done properly beats five genres done badly.
- **Build lean** — no service, library, or abstraction gets added unless it solves a real problem we have right now.
- **One source of truth** — this document. If it's not in here, it doesn't exist yet.

3. Target Users

Persona	Who	Core Need
The Curious Fan	Loves the music, misses meaning	Understand lyrics; connect with the art
The Parent	Teen's parent hearing SDK at home	Know what their kid listens to, without judgment
The International Listener	Non-SA audience via streaming	Context and vocab to appreciate the music
The Journalist	Media covering SA culture	Accurate, citable cultural translations
The Artist	Emerging SDK artists	See how their work lands with an outside audience

4. Core Features

4.1 Input Methods

- **Typed Lyrics** — paste raw lyrics, hit translate. No upload, no processing delay. Fastest path.
- **Audio Upload** — MP3, WAV, M4A up to 50 MB. Whisper transcribes to text, user can edit before translating.
- **YouTube Link** — paste a URL. System fetches auto-captions first; falls back to audio extraction via yt-dlp if unavailable. Cached instantly if song is already in the library.

4.2 4-Panel Translation Output

Panel	What It Shows
Plain Translation	Full song rewritten in natural, standard English. No jargon. Reads like a translated poem.
Line-by-Line	Original line alongside English translation. Slang terms highlighted; tap/hover for KB definition.
Slang Dictionary	Auto-extracted glossary: term, definition, origin note, example, confidence level.
Cultural Context	2–4 paragraph AI-written narrative: who's speaking, to whom, about what, and why it matters culturally.

4.3 Slang Knowledge Base (KB)

The KB is the core differentiator. It is a growing dictionary of SDK-specific terms that powers all translation outputs and gets smarter over time.

- **Phase 1 — AI Seeding:** Every submitted song runs through Claude which extracts unknown terms and generates provisional definitions. These are stored as candidates.
- **Phase 2 — Moderation:** We (the team) review candidates in an internal admin queue and approve, edit, or reject them. Approved entries enter the live KB.
- **Phase 3 — Community:** Once we have users, community members can flag wrong entries and submit new ones. All go through moderation.
- **Ongoing:** Every new song adds new candidates. The KB never stops growing.

4.4 Public Song Library

Searchable library of already-translated songs. Browse by artist or recency. Each entry shows all 4 panels and a community rating. No account needed to browse or view.

4.5 Sharing

Every translated song gets a unique URL (`/s/[song-id]`). Share button copies to clipboard. Shared pages are fully public.

5. User Stories

Format: *As a [persona], I want to [action], so that [outcome].* Acceptance criteria define done.

5.1 Typed Lyrics Input

US-01 — Paste and Translate

As a fan, **I want to** paste lyrics and hit translate, **so that** I get a full breakdown instantly.

Done when:

- Accepts 1–500 lines of text.
- Translate button disabled until at least 1 non-empty line present.
- All 4 panels populated within 10 seconds for lyrics under 50 lines.
- No account or login required at any point.

US-02 — Clear and Re-translate

As a user, **I want to** clear input and paste new lyrics, **so that** I can translate multiple songs in one session.

Done when:

- Clear button resets input and all 4 output panels.
- No page reload required.

5.2 Audio Upload

US-03 — Upload Audio File

As a fan, **I want to** upload an audio file, **so that** I don't have to type anything.

Done when:

- Accepts MP3, WAV, M4A only. Other formats rejected with clear message.
- Files over 50 MB rejected before upload begins.
- Progress bar shown during upload.
- Transcribed lyrics displayed in an editable area before translation.
- User can fix transcription errors then hit translate.
- All 4 panels populated on success.

US-04 — Edit Transcription Before Translating

As a user, **I want to** fix auto-transcribed lyrics, **so that** transcription errors don't break the translation.

Done when:

- Transcribed text shown in editable textarea.
- "Looks good, translate" button proceeds.
- Edits reflected in the final output.

5.3 Streaming Link

US-05 — Translate via YouTube Link

As a user, **I want to** paste a YouTube URL and get a translation, **so that** I don't need to download anything.

Done when:

- Accepts `youtube.com/watch?v=` and `youtu.be/` formats.
- Fetches auto-captions first; falls back to yt-dlp audio extraction.
- If song already in library, cached result served instantly.
- User sees source method: "from captions" or "from audio."

US-06 — Reject Non-SDK Content

As a user submitting a non-SDK song, **I want to** be clearly told the scope, **so that** I'm not confused by a bad result.

Done when:

- High-confidence non-SDK (>0.85) shows a friendly genre warning, not an error. User can override and proceed anyway.
- Low-confidence detection proceeds with a disclaimer banner.

5.4 Translation Output

US-07 — View Plain Translation

As a parent, **I want to** read the song in plain English, **so that** I understand it with zero prior knowledge.

Done when:

- Full song in standard English prose, no jargon.
- Reads naturally — not word-for-word literal.
- Default panel shown after translation completes.

US-08 — View Line-by-Line Breakdown

As a fan, **I want to** see each line beside its translation, **so that** I learn which words mean what.

Done when:

- Original left / top, translation right / bottom. Side-by-side desktop, stacked mobile.
- Slang terms highlighted in brand purple.
- Tap/hover shows inline tooltip with KB definition.
- Unknown terms show: "Not in our dictionary yet — help us define it!"

US-09 — View Slang Dictionary

As a user, **I want to** see a glossary of all slang, **so that** I learn the vocab for future songs.

Done when:

- All flagged terms listed alphabetically.
- Each entry: term, definition, origin note, example, confidence badge.
- Unverified terms visually distinguished.
- Clicking a term highlights it in the line-by-line panel.

US-10 — View Cultural Context

As a journalist, **I want to** read a cultural narrative, **so that** I understand the song's story and significance.

Done when:

- 2–4 paragraph narrative covering who, what, to whom, and why.
- Written in accessible, engaging prose.
- Labelled: "AI-generated — community review pending."

US-11 — Switch Output Panels

As a user, **I want to** switch between the 4 panels, **so that** I explore at my own pace.

Done when:

- 4 tabs: Translation, Line by Line, Slang Dictionary, Cultural Context.
- Switching is instant — no reload.
- Desktop toggle: "Show all panels" displays 2×2 grid.

5.5 Knowledge Base & Community

US-12 — Flag a Wrong Definition

As a native speaker, **I want to** flag a bad KB entry, **so that** it gets corrected.

Done when:

- Flag icon on every KB term. No account to flag.
- Form: "What's wrong?" with text input.
- 3+ unresolved flags auto-downgrade entry to Unverified.
- Flags appear in admin moderation queue.

US-13 — Submit a Missing Definition

As a community member, **I want to** add a missing term, **so that** the KB grows.

Done when:

- Contribute form: Term, Definition, Origin note, Example sentence.
- Stored as KBCandidate — not live until approved by us.
- Confirmation: "Submitted! We'll review it."

5.6 Library & Social

US-14 — Browse Song Library

As a user, **I want to** browse translated songs, **so that** I can explore without submitting anything.

Done when:

- Song cards: title, artist, date, avg rating.
- Default sort: most recently translated. Filter by artist text search.
- No account required to browse.

US-15 — Rate a Translation

As a user, **I want to** rate accuracy, **so that** the best translations surface.

Done when:

- 1–5 star widget. No account needed (session-based dedup).
- Avg rating and count displayed.
- Songs below 3 stars flagged in admin queue.

US-16 — Share a Translation

As a user, **I want to** share a translation link, **so that** others can view it without submitting the song.

Done when:

- Every song gets a unique URL: `/s/[song-id]`.

- Share button copies URL with "Copied!" toast.
- Shared pages publicly accessible, no account needed.

6. Error States

No raw error codes ever shown to users. Every error is human and helpful.

Scenario	User Message	System Behaviour
File too large (>50 MB)	"File is too big (max 50 MB). Trim the audio or paste lyrics instead."	Reject before upload.
Bad file format	"We only support MP3, WAV, and M4A."	Reject at file picker.
Whisper fails	"Couldn't transcribe this. Try a cleaner recording or paste lyrics manually."	Log error; offer lyrics fallback.
Geo-blocked link	"Can't access that link in your region. Paste the lyrics instead."	Offer lyrics fallback.
Invalid / dead URL	"That link doesn't work. Check the URL and try again."	Validate before processing.
Translation timeout	"Taking longer than expected. Try again in a moment."	Auto-retry once; message on 2nd fail.
Non-SDK detected	"Doesn't look like SDK music — we specialise in SDK for now. Try anyway?"	Warning + override button.
Empty lyrics	"Paste some lyrics first!"	Disable button; inline hint.
Zero KB matches	"New territory for us — translation provided but dictionary is empty. Help us build it!"	Proceed; show contribute CTA.
Offline	"You're offline. Check your connection."	Detect offline; pause processing.

7. Data Models

> These are the Prisma schema definitions we build from directly. Field names here are the field names in the code.

7.1 Song

```
model Song {
  id                String        @id @default(uuid())
  title             String?
  artist            String?
  sourceType        SourceType    // TYPED_LYRICS | AUDIO_UPLOAD |
    YOUTUBE_LINK
  sourceUrl         String?
  rawLyrics         String        @db.Text
  genreTag          GenreTag      @default(SDK)
  genreConfidence   Float         @default(1.0)
  isPublic          Boolean       @default(true)
  avgRating         Float?
  ratingCount       Int           @default(0)
  createdAt         DateTime      @default(now())
  updatedAt         DateTime      @updatedAt
  translation       Translation?
  ratings           Rating[]
}

enum SourceType { TYPED_LYRICS  AUDIO_UPLOAD  YOUTUBE_LINK }
enum GenreTag   { SDK  UNKNOWN }
```

7.2 Translation

```
model Translation {
  id                 String   @id @default(uuid())
  songId             String   @unique
  song               Song     @relation(fields: [songId], references: [
    id])
  plainTranslation   String   @db.Text
  lineByLine         Json
  // lineByLine shape:
  // [{ original: string, translation: string, flaggedTerms: string[]
    }]
  culturalContext    String   @db.Text
  aiModelVersion     String   // e.g. "gemini-2.5-flash" or "llama
    -3.3-70b"
  confidenceOverall Float
  processingTimeMs   Int
```

```
  createdAt         DateTime @default(now())
}
```

7.3 Job (Async Processing Queue)

```
model Job {
  id          String    @id @default(uuid())
  type        JobType   // TRANSCRIBE | TRANSLATE | FULL_PIPELINE
  status      JobStatus @default(PENDING)
  payload     Json      // { sourceType, lyrics?, fileKey?, url? }
  songId      String?   // populated when complete
  errorMsg    String?
  attempts    Int       @default(0)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

enum JobType   { TRANSCRIBE  TRANSLATE  FULL_PIPELINE }
enum JobStatus { PENDING  PROCESSING  COMPLETE  FAILED }
```

7.4 KBEntry

```
model KBEntry {
  id              String      @id @default(uuid())
  term            String      @unique
  definition      String      @db.Text
  originNote      String?
  dialectTag      String?
  exampleSentence String?     @db.Text
  confidence      Confidence  @default(UNVERIFIED)
  source          KBSource    @default(AI_SEEDED)
  flagCount       Int         @default(0)
  isApproved      Boolean     @default(false)
  createdAt       DateTime    @default(now())
  updatedAt       DateTime    @updatedAt
  flags           Flag[]
}

enum Confidence { HIGH  MEDIUM  UNVERIFIED }
enum KBSource   { AI_SEEDED  COMMUNITY_SUBMITTED  MANUALLY_CURATED }
```

7.5 KBCandidate

```
model KBCandidate {
  id              String          @id @default(uuid())
  term            String
  definition      String          @db.Text
```

```
  originNote      String?
  exampleSentence String?
  submittedByIp   String
  status          CandidateStatus @default(PENDING)
  reviewedAt      DateTime?
  createdAt       DateTime        @default(now())
}

enum CandidateStatus { PENDING  APPROVED  REJECTED }
```

7.6 Flag

```
model Flag {
  id          String   @id @default(uuid())
  kbEntryId   String
  kbEntry     KBEntry  @relation(fields: [kbEntryId], references: [id])
  reason      String   @db.Text
  submittedIp String
  resolved    Boolean  @default(false)
  createdAt   DateTime @default(now())
}
```

7.7 Rating

```
model Rating {
  id        String   @id @default(uuid())
  songId    String
  song      Song     @relation(fields: [songId], references: [id])
  score     Int      // 1-5
  sessionId String
  createdAt DateTime @default(now())

  @@unique([songId, sessionId]) // one rating per session per song
}
```

8. API Routes

All routes live under `/app/api/` in the Next.js App Router. All responses are JSON. Errors follow: `{"error": {"code": string, "message": string}}`.

8.1 Translation

8.2 Songs

```
GET    /api/songs/[songId]
       Returns: { song: Song, translation: Translation, kbEntries:
   KBEntry[] }

GET    /api/songs
       Query:   ?sort=recent|rating&search=string&page=1&limit=20
       Returns: { songs: Song[], total: number, page: number }
```

8.3 Knowledge Base

```
GET    /api/kb/[term]
       Returns: KBEntry | 404

POST   /api/kb/flag
       Body:    { kbEntryId: string, reason: string }
       Returns: { success: true }

POST   /api/kb/contribute
       Body:    { term, definition, originNote?, exampleSentence? }
       Returns: { success: true, candidateId: string }
```

8.4 Ratings

```
POST   /api/songs/[songId]/rate
       Body:    { score: number (1-5), sessionId: string }
       Returns: { avgRating: number, ratingCount: number }
```

9. Folder Structure

```
decoded-sound/
 app/
    page.tsx                    # Landing + input (home)
    s/[songId]/page.tsx         # Shared translation view
    library/page.tsx            # Public song library
    api/
       translate/
          lyrics/route.ts
          audio/route.ts
          link/route.ts
       jobs/[jobId]/route.ts
       songs/
          route.ts            # GET /api/songs (library)
```

```
          [songId]/
              route.ts       # GET /api/songs/[songId]
              rate/route.ts
      kb/
          [term]/route.ts
          flag/route.ts
          contribute/route.ts
components/
   input/
      LyricsInput.tsx
      AudioUpload.tsx
      LinkInput.tsx
   output/
      OutputTabs.tsx
      PlainTranslation.tsx
      LineByLine.tsx
      SlangDictionary.tsx
      CulturalContext.tsx
   kb/
      TermTooltip.tsx
      FlagForm.tsx
      ContributeForm.tsx
   library/
      SongCard.tsx
      LibraryGrid.tsx
   ui/
       Toast.tsx
       StarRating.tsx
       ProgressBar.tsx
lib/
   pipeline/
      transcribe.ts          # Whisper API call
      translate.ts           # Claude API call
      extractYoutube.ts      # yt-dlp wrapper
      detectGenre.ts         # Genre classification
   db/
      prisma.ts              # Prisma client singleton
   jobs/
      enqueue.ts             # Write job to DB
      poll.ts                # Update job status
      worker.ts              # Process pending jobs
   prompts/
       translationPrompt.ts   # Claude system prompt builder
prisma/
   schema.prisma              # All models from Section 7
types/
   index.ts                   # Shared TS types
.env.local                     # API keys (never committed)
package.json
```

10. AI Pipeline — Step by Step

1. **Receive file** → validate type and size → upload to temp storage → create Job record (`status: PENDING`).
2. **Worker picks up job** → sets `status: PROCESSING`.
3. **Transcription** → call Whisper transcription (via Groq Whisper endpoint — free, same API shape as OpenAI) → receive raw text → store on Job payload.
4. **Show to user for edit** → frontend polls job, sees `status: AWAITING_EDIT` → shows editable textarea. User confirms or edits → sends corrected lyrics back.
5. **Preprocess** → split into lines → tag potential slang candidates via regex + known-term pattern matching.
6. **KB Lookup** → query KBEntry table for all candidate terms → build context string of known definitions to pass to Gemini.
7. **Genre detection** → Groq (Llama 3.3 70B) classifies SDK vs non-SDK with confidence score:
   - >0.85 non-SDK: return genre warning, offer override.
   - <0.6 SDK confidence: proceed with disclaimer banner.
   - ≥ 0.6 SDK: proceed normally.
8. **Gemini translation call** → structured JSON prompt (see Section 10.2) → Gemini 2.5 Flash receives lyrics + KB context → returns all 4 panels + unknown terms.
9. **Term extraction** → save unknown terms as KBCandidate records with Gemini's provisional definitions, status `PENDING`.
10. **Confidence scoring** → calculate per-term and overall score based on KB hit rate and Gemini self-reported certainty.
11. **Persist** → create Song + Translation records → set Job `status: COMPLETE, songId: [id]`.
12. **Frontend** → polls job → sees COMPLETE → fetches `/api/songs/[songId]` → renders all 4 panels.

Typed lyrics path skips steps 1–4. Starts at step 5 directly.

YouTube path replaces step 3 with: fetch auto-captions via YouTube API. If unavailable, run yt-dlp to extract audio, then Groq Whisper. If song already in library, skip entire pipeline and serve cached result.

10.1 Job Worker Design

The job queue is a Postgres table (the `Job` model). The worker is a Next.js API route that gets triggered on a schedule:

```
// lib/jobs/worker.ts
// Called by: GET /api/worker (hit by a Vercel cron job every 30s)
// 1. Query: SELECT * FROM Job WHERE status = PENDING ORDER BY
    createdAt LIMIT 5
// 2. For each job: set status = PROCESSING, run pipeline, set COMPLETE
     or FAILED
// 3. On failure: increment attempts. After 3 attempts: set status =
    FAILED.
```

Vercel cron (free tier) supports cron jobs with a minimum interval of 60 seconds on the free plan. For v1.0 this is fine. If we need faster processing, we switch to a persistent worker on Railway later.

10.2 Gemini Translation Prompt

```
SYSTEM:
You are DecodedSound  a cultural translation engine specialised in SDK
(esdeekid) music from the Cape Flats, South Africa. You have deep
    knowledge
of the dialect, phonetic wordplay, Afrikaans and Xhosa borrowings, and
    the
cultural references of this genre.

You will receive:
- Raw song lyrics
- A KB context block of known terms and definitions

Produce a single JSON object with exactly these keys:
{
  "plainTranslation": string,       // full song in plain English,
    natural prose
  "lineByLine": [                   // one object per lyric line
    {
      "original": string,
      "translation": string,
      "flaggedTerms": string[]      // terms from this line that are in
     KB or unknown
    }
  ],
  "culturalContext": string,        // 2-4 paragraph narrative,
    accessible prose
  "unknownTerms": [                 // terms NOT in KB
    {
      "term": string,
      "provisionalDefinition": string,
      "confidence": number          // 0.0 - 1.0
    }
  ],
```

```
  "genreConfidence": number,        // 0.0 - 1.0 SDK genre match
  "overallConfidence": number       // 0.0 - 1.0 translation confidence
}

Rules:
- Never hallucinate definitions. If a term is unknown, include it in
  unknownTerms with your best guess and a low confidence score.
- Flag uncertainty. Never present a guess as a fact.
- Output strict JSON only. No markdown, no preamble, no explanation.
- Cultural context must be readable by someone with zero knowledge of
    SDK.
- Plain translation must read naturally  not word-for-word literal.
```

11. AI Provider Strategy

DecodedSound uses a **multi-provider, zero-cost AI stack** for v1.0. All providers use OpenAI-compatible APIs — swapping providers means changing two lines of code, nothing else.

Task	Provider	Model	Free Limit
Translation, line-by-line, cultural context	Gemini (existing key)	gemini-2.5-flash	250k TPM, 250 RPD
Genre detection	Groq	llama-3.3-70b	14,400 req/day
KB term extraction	Groq	llama-3.3-70b	14,400 req/day
Audio transcription	Groq Whisper	whisper-large-v3	7,200 audio-sec/hr
Fallback (any provider)	OpenRouter	free tier models	50 req/day (free)

11.1 Why This Split

- **Gemini for translation** — you already have the key, the 1M token context window handles long songs easily, and Gemini 2.5 Flash has strong multilingual and cultural reasoning for the quality we need.
- **Groq for detection and transcription** — genre classification and term extraction are simpler tasks where speed matters more than nuance. Groq's Llama 3.3 70B is fast and the free tier is the most generous of any provider (14,400 req/day). Groq also offers a free Whisper endpoint so transcription costs nothing.
- **OpenRouter as fallback** — one key that routes to 30+ models. If Gemini or

Groq hit daily limits, we reroute automatically without downtime. No credit card needed until we scale.

11.2 Provider Abstraction Layer

All AI calls go through a single abstraction in `lib/ai/client.ts`. Swapping providers never touches business logic:

```
// lib/ai/client.ts
import OpenAI from 'openai' // openai SDK works for all providers below

export const gemini = new OpenAI({
  baseURL: 'https://generativelanguage.googleapis.com/v1beta/openai/',
  apiKey: process.env.GEMINI_API_KEY,
})

export const groq = new OpenAI({
  baseURL: 'https://api.groq.com/openai/v1',
  apiKey: process.env.GROQ_API_KEY,
})

export const openrouter = new OpenAI({
  baseURL: 'https://openrouter.ai/api/v1',
  apiKey: process.env.OPENROUTER_API_KEY,
})

// Usage in pipeline:
// Translation  -> gemini.chat.completions.create({ model: 'gemini-2.5-
    flash', ... })
// Genre detect -> groq.chat.completions.create({ model: 'llama-3.3-70b-
    versatile', ... })
// Fallback     -> openrouter.chat.completions.create({ model: 'meta-
    llama/llama-3.3-70b-instruct:free', ... })
```

11.3 When We'd Pay

We stay fully free until one of these triggers:

- Gemini free RPD (250/day) is consistently maxed out — upgrade to Gemini pay-as-you-go ($0.075 per 1M tokens for Flash).
- Groq free limits hit — Groq paid tier is still very cheap.
- At meaningful scale, we evaluate whether a single premium provider (Claude, GPT-4o) gives better quality ROI than the multi-provider setup.

12. Tech Stack — Final Decisions

Layer	Technology	Why
Framework	Next.js 14 (App Router)	Fullstack in one repo — frontend + API routes. No separate backend.
Language	TypeScript	Type safety across DB, API, and UI. Catches bugs before runtime.
Database	Postgres (local dev) → Supabase (production)	Local is free and fast. Supabase free tier when we go live — also handles file storage, killing S3.
ORM	Prisma	Type-safe queries, migrations built in, plays perfectly with TypeScript and Supabase.
Transcription	OpenAI Whisper API	Best multilingual transcription. Dialect-sensitive. API call — no Python needed.
Translation AI	Anthropic Claude API (claude-sonnet-4-6)	Contextual nuance, slang, cultural storytelling.
YouTube Extraction	yt-dlp (CLI, called from API route)	Industry standard for YouTube audio. Free.
Styling	Tailwind CSS	Fast, responsive, no CSS files to manage.
Deployment	Vercel (free tier)	Zero config. Deploys on every push. Free cron jobs for the job worker.
Job Queue	Postgres Job table + Vercel cron	No extra service. We control it. Scales fine for v1.0.
File Storage	Supabase Storage (when live)	Replaces S3 entirely. Same free tier account as the DB.
Auth	Not in v1.0	Clerk is ready to add in v1.5 when accounts are needed.

What we removed from v1: AWS Lambda, AWS SQS, AWS S3, AWS RDS, FastAPI, separate Python backend. All replaced by Next.js API routes + Postgres + Supabase Storage. Same capability, a fraction of the complexity.

13. Environment Variables

```
# .env.local (never commit this file)

# Database
DATABASE_URL="postgresql://localhost:5432/decodedsound"

# AI APIs
GEMINI_API_KEY=""          # Gemini 2.5 Flash  primary translation
GROQ_API_KEY=""            # Groq  genre detection, term extraction,
    Whisper transcription
OPENROUTER_API_KEY=""      # OpenRouter  fallback if primary providers
    hit limits

# Supabase (add when going live, leave blank in local dev)
NEXT_PUBLIC_SUPABASE_URL=""
NEXT_PUBLIC_SUPABASE_ANON_KEY=""
SUPABASE_SERVICE_ROLE_KEY=""

# App
NEXT_PUBLIC_APP_URL="http://localhost:3000"
WORKER_SECRET=""           # secret to protect /api/worker cron
    endpoint
```

14. Milestone Plan

Week	Milestone	Deliverables
1–2	Foundation	Repo, Prisma schema + local DB, Next.js shell, Tailwind setup, `.env.local`, Whisper API test call, Claude API test call.
3–4	Core Pipeline	Typed lyrics → Claude → 4-panel output working end-to-end. Job table + worker. KB seeded with first 200 terms.
5–6	Audio & YouTube	Audio upload with Whisper + edit step. YouTube link with caption fetch + yt-dlp fallback. Caching for known songs.
7–8	KB & Community	KB lookup in pipeline. Confidence scoring. Flag form. Contribute form. Admin moderation queue (basic internal page).
9–10	Library & Sharing	Public song library. Search by artist. Star ratings. Shareable URLs. Genre detection + mismatch warning.
11–12	Polish & Errors	All error states. Mobile-responsive UI. Full error logging. Performance pass. Accessibility basics.
13–14	Beta & Launch	Internal beta. KB at 500+ terms. Bug fixes. Vercel deploy. Public launch.

15. MVP Scope

In v1.0

- Typed lyrics, audio upload, YouTube link input
- 4-panel output: plain translation, line-by-line, slang dictionary, cultural context
- Genre detection with SDK-only scope and friendly warnings
- Slang KB: 500+ AI-seeded terms, flag + contribute forms
- Admin moderation queue (internal, not public-facing)
- Public song library with search and ratings
- Shareable URLs per song
- Fully open — no account required for anything

- Web only, English UI

Explicitly NOT in v1.0

- User accounts or authentication
- Mobile application
- Any genre other than SDK / esdeekid
- Non-English UI
- SoundCloud or Spotify audio extraction
- PDF / image export
- Embeddable widget or public API
- Monetization of any kind
- Artist dashboard or analytics
- English → SDK reverse translation (Phase 2)

Phase 2 — v1.5 (Weeks 14–24)

- **English → SDK Reverse Translation** — user types plain English and gets SDK-style output back. Fun, viral, drives engagement. Powered by the same KB in reverse. See Section 16 for full feature spec.
- SoundCloud and Spotify integration
- User accounts (optional — for saved history)
- Community KB voting and contributor leaderboard
- Artist song-claim and official commentary
- PDF / image export of translated lyrics
- Access limits if needed (daily cap for guests)
- Analytics dashboard (internal)
- Monetization decision and implementation begins

Phase 3 — v2.0 (Month 6+)

- Mobile application (React Native)
- Genre expansion: Amapiano, Gqom, Kwaito (separate KB per genre)
- Multilingual UI: Afrikaans, Zulu, French
- Embeddable widget for external sites

- Public API (B2B licensing)
- Real-time lyric translation

16. Feature Spec: English → SDK Reverse Translation

Inspired by: esdictionary.com (The Right Note, YouTube). Their tool is a static lookup table — ours uses the live KB + AI to generate contextually authentic SDK-style output, not just word swaps.

The reverse translation mode lets users type plain English and receive SDK-style output — the same language, slang, and phonetic style that EsDeeKid uses. This is the feature that makes DecodedSound *fun* and shareable, not just educational.

16.1 User Stories — Reverse Mode

US-17 — Translate English to SDK Style

As a fan, **I want to** type something in plain English and get it back in SDK style, **so that** I can engage with the culture and share it with friends.

Done when:

- A toggle on the input switches between "SDK → English" and "English → SDK" modes. Default is SDK → English.
- In reverse mode, user types plain English text (up to 500 words).
- Output is a single panel: the SDK-style rewrite.
- Output uses real KB terms where appropriate — not invented slang.
- AI is instructed to match EsDeeKid's phonetic style, cadence, and vocabulary as closely as possible.
- A disclaimer is shown: "AI-generated SDK style — not verified by native speakers."

US-18 — Share a Reverse Translation

As a user, **I want to** share my SDK-style output, **so that** I can post it or send it to friends.

Done when:

- Reverse translations get a shareable URL like forward translations.
- A "Copy text" button copies just the SDK output for pasting into messages or social media.
- Output is clearly labelled as AI-generated SDK style.

16.2 How It Works — Reverse Pipeline

1. User types plain English text and selects "English → SDK" mode.
2. System queries KB for high-confidence SDK terms relevant to the input's semantic content.
3. Gemini receives: plain English input + KB context of real SDK terms + system prompt instructing it to rewrite in authentic EsDeeKid style.
4. Output is a single SDK-style rewrite using real KB vocabulary where possible, AI-inferred style where not.
5. Result stored with `sourceType: REVERSE_TRANSLATION` in DB.
6. Shareable URL generated. Disclaimer banner shown.

16.3 Gemini Prompt — Reverse Mode

```
SYSTEM:
You are DecodedSound in reverse mode. Your job is to rewrite plain
    English
text in the style of EsDeeKid (SDK)  a Cape Flats hip-hop artist from
South Africa.

EsDeeKid's style includes:
- Cape Flats slang and dialect
- Afrikaans words and phonetic borrowings
- Short, punchy sentences with street cadence
- Phonetic wordplay and double meanings
- The following verified KB terms (use these where natural): {kb_terms}

Rules:
- Only use KB terms where they fit naturally. Do not force them.
- Do not invent slang that isn't in the KB  use AI judgement for style,
  not vocabulary.
- Keep the core meaning of the input intact.
- Output the SDK rewrite only. No explanation, no preamble.
```

16.4 DB Change for Reverse Mode

One addition to the `SourceType` enum:

```
enum SourceType {
  TYPED_LYRICS
  AUDIO_UPLOAD
  YOUTUBE_LINK
  REVERSE_TRANSLATION   // new in v1.5
}
```

16.5 Competitive Advantage Over esdictionary.com

Feature	esdictionary.com	DecodedSound
Translation method	Static word lookup	AI + live KB context
Handles sentences	No — word by word	Yes — full sentence rewrite
KB grows over time	No — frozen	Yes — every submission adds terms
Phonetic style matching	No	Yes — Gemini mimics cadence
Forward translation	Yes	Yes (core feature)
Cultural context	No	Yes
Audio / YouTube input	No	Yes

17. Success Metrics

Metric	Target	How Measured
Translation accuracy (user-rated)	≥85% good	Post-translation star rating
Monthly active users	5,000 by month 3	Vercel analytics
Unique songs translated	1,000 in library	DB count
Avg session duration	>4 minutes	Analytics
KB terms at launch	500+	DB count
KB terms at 6 months	2,000+	DB count
Community contributions/month	100+ by month 3	Submission logs
Shares/month	500+ by month 2	URL click tracking

18. Risks & Mitigations

Risk	Likelihood	Mitigation
Whisper inaccuracy on fast delivery	High	User edit step before translation. Post-launch fine-tuning.
Slang evolves faster than KB	High	AI extracts candidates from every submission. Community validates.
Cultural misrepresentation	Medium	Confidence scoring. Flagging. "AI-generated" label on all outputs.
Claude hallucination	Medium	Explicit prompt instruction: flag unknown, never guess as fact.
Copyright on YouTube extraction	Medium	Captions-first approach. yt-dlp as fallback only.
Low community volume early	Medium	AI-first KB means launch doesn't depend on community.
Abuse of open flag/contribute	Low–Med	IP rate limiting. Honeypot fields. Moderation queue.
Scope creep to other genres	Low	Hard line in this document. SDK only until v1.0 ships.

19. Open Questions

1. **Monetization** — Must be decided before v1.5 architecture. Options: freemium (guest cap, paid unlimited), B2B API, ads. Current architecture does not block any of these.
2. **Artist rights / DMCA** — Do artists have the right to remove their song from the public library? We need a policy before launch.
3. **Content moderation policy** — Explicit content and illegal references will appear in SDK music. What is the library policy? Recommended: no filter in v1.0, community flagging handles it.
4. **Launch partnerships** — SoundCloud, Audiomack, or SA music blogs as launch partners for initial KB seeding and user acquisition.

DecodedSound

Making the culture accessible to the world.

Full PRD + Engineering Spec v3.2 | February 2026 | SDK only (v1.0) | Confidential